

Teacher-Guided Recovery for GETA-Based Joint Structured Pruning and Quantization: A Comparative Study of Fixed and Compression-Aware Distillation

MD. Sabbir Alam Pial^{a,*}, Ayesha Akter Lima^a, Md. Abu Raihan^a, Al Reyad^a, Md. Muktar Hossain^a

^aDepartment of Computer Science and Engineering, Varendra University, Rajshahi, Bangladesh

Abstract

Model compression is critical for deploying deep neural networks on resource constrained devices. This study compares GETA-only training, fixed vanilla knowledge distillation (KD), and Compression-Aware Knowledge Regularization (CAKR), an adaptive method that controls the teacher signal using compression state and confidence information, on top of GETA-based joint structured pruning and quantization. Experiments cover nine architecture-dataset combinations (ResNet20, ResNet56, VGG7 on CIFAR-10, CIFAR-100 and TinyImageNet-200) with paired significance testing against GETA-only. KD based recovery offers limited benefit on easier CIFAR-10 tasks but larger, more often significant gains on harder datasets, reaching $p < 0.05$ in three of nine settings. CAKR reaches significance only on ResNet20/CIFAR-100 ($p = 0.031$) and most reduces cross-seed variability on ResNet20/CIFAR-10 (from ± 0.61 to ± 0.13), but falls below GETA-only on ResNet20/TinyImageNet-200, where fixed KD instead shows its largest gain. These results reject a simple rule where adaptive KD favors smaller students and fixed KD favors larger ones; effectiveness depends on the joint interaction of model capacity, dataset difficulty, and compression severity.

*Corresponding author

Email addresses: sabbir.vu11@gmail.com (MD. Sabbir Alam Pial), ayeshalima999@gmail.com (Ayesha Akter Lima), raihan.str13@gmail.com (Md. Abu Raihan), reyad.vu157@gmail.com (Al Reyad), mmukhtar997@gmail.com (Md. Muktar Hossain)

Keywords: model compression, quantization, KD, GETA, CAKR

1. Introduction

Deep neural networks have achieved strong performance in computer vision tasks, but their computational and memory requirements limit deployment on resource-constrained devices. ResNet56 on CIFAR-100, for example, requires 861K parameters and approximately 125M multiply-accumulate operations per inference - well beyond typical edge device budgets [1, 2]. Model compression is therefore an important step between training and deployment [3, 4]. Among compression techniques, structured pruning and quantization are widely adopted because they reduce both model size and computational cost [5].

Structured pruning removes hardware-friendly components such as channels, filters, or groups, directly reducing model width and multiply-accumulate operations. Quantization reduces numerical precision, lowering storage and inference cost [4, 6, 7]. In practice, these techniques are often applied separately, but this can be suboptimal because pruning changes the network structure and may increase sensitivity to low-precision representation. Joint pruning and quantization addresses this by optimizing sparsity and bit-width under a unified process [8, 9].

GETA provides an automatic framework for joint structured pruning and quantization-aware training, with explicit control over target sparsity and bit-width constraints [9]. Although GETA is a strong compression backbone, the compressed student is primarily optimized with cross-entropy. During aggressive joint compression, the student changes continuously as structured groups are removed and quantization parameters are updated. Teacher guidance through knowledge distillation can help recover lost information, but the best form of teacher guidance is not obvious for this setting.

We studied teacher-guided recovery for GETA-based joint compression by comparing three approaches: GETA-only (cross-entropy only), fixed vanilla KD, and CAKR, a compression-aware adaptive regularizer that controls the teacher signal using the current compression state and teacher reliability. Prior work has combined KD with pruning and quantization [10, 11, 12], but none directly addresses fixed versus adaptive KD within GETA-based joint structured pruning and quantization, evaluated with paired statistical testing across multiple architectures and datasets. To the best of our knowledge, no prior study has systematically compared GETA-only, fixed KD, and

35 compression-aware KD under the same GETA-based joint pruning and
quantization protocol across nine architecture-dataset combinations with
significance testing.

The main contributions of this paper are summarized as follows:

- 40 • Teacher-guided recovery for GETA-based joint structured pruning and
weight-only quantization is investigated under a common compression
protocol.
- GETA-only, GETA with fixed vanilla KD, and GETA with CAKR
are evaluated across nine architecture-dataset combinations involving
ResNet20, ResNet56, VGG7 on CIFAR-10, CIFAR-100, TinyImageNet-200,
45 each over three seeds, with paired t -test significance and Cohen’s d effect
size reported for every comparison against the GETA-only baseline.
- CAKR is introduced as a compression-aware KD strategy in which the
teacher signal is adaptively controlled using stable-sparsity activation,
teacher-correct confidence weighting, and compression-state indicators,
50 while cross-entropy is retained as the primary objective.
- Of the eighteen method-setting comparisons made in this study, four
reach statistical significance ($p < 0.05$): one for CAKR (ResNet20/CIFAR-100,
 $p = 0.031$) and three for fixed vanilla KD (ResNet56/CIFAR-100,
VGG7/CIFAR-10, and VGG7/TinyImageNet-200).
- 55 • CAKR’s only statistically significant improvement is observed on ResNet20
with CIFAR-100, where accuracy increases from 50.91% to 51.74% ($p =$
0.031, Cohen’s $d = 3.19$). CAKR also demonstrates improved cross-seed
stability on ResNet20/CIFAR-10, where the standard deviation is
reduced from ± 0.61 for GETA-only to ± 0.13 , independent of whether
60 the corresponding mean-accuracy gain reaches significance.
- Fixed vanilla KD provides the most frequent statistically significant
recovery among the two methods, with significant improvements in
three of the nine evaluated settings, spanning both a higher-capacity
architecture (ResNet56) and a mid-sized one (VGG7). Its largest
65 significant gain is observed on ResNet56/CIFAR-100 (+0.91 percentage
points, $p = 0.011$).

- On TinyImageNet-200, the highest mean accuracy is achieved by fixed KD for all three architectures. A significant gain is obtained only for VGG7 ($p = 0.020$), whereas the gains for ResNet20 ($p = 0.173$) and ResNet56 ($p = 0.083$) do not reach significance; CAKR does not improve over GETA-only at all on ResNet20/TinyImageNet-200.
- The results do not support a simple architecture-size rule in which adaptive KD is preferable for small models and fixed KD is preferable for large ones. Instead, the relative effectiveness of fixed and adaptive distillation is shown to depend on the joint interaction among student capacity, dataset difficulty, and compression severity, and this interaction is examined mechanistically through CAKR’s own compression-risk gating behaviour.
- As a supplementary analysis (Section 6.4), an interpolated Hinton-style KD blend is compared against the additive KD formulation used throughout the main experiments on TinyImageNet-200; the interpolated form yields larger gains and reaches significance in two of the three architectures, versus one of three for the additive form, suggesting that KD loss formulation itself affects recoverability independent of architecture and dataset difficulty.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 provides background on GETA and KD. Section 4 describes the compared teacher-guided recovery methods. Section 5 presents the experimental setup. Section 6 reports results and discussion. Section 7 concludes the paper.

2. Related Work

2.1. Structured Pruning and Dependency-Aware Compression

Structured pruning removes channels, filters, or dependency-connected groups to obtain hardware-friendly acceleration [3, 5]. Unlike unstructured pruning, structured pruning directly reduces dense tensor dimensions and multiply-accumulate operations. Channel pruning and filter pruning methods remove components based on importance criteria such as magnitude or activation statistics [5]. DepGraph automates structured pruning by tracing structural dependencies between layers, enabling group-wise pruning across diverse architectures [13]. OTO and OTOv2 further integrate sparsity-inducing

optimization directly into training, reducing the need for separate prune-and-fine tune cycles [14, 15]. These dependency-aware methods motivate the structured group view used by GETA for joint pruning and quantization.

2.2. Quantization and Quantization-Aware Training

105 Quantization reduces storage and arithmetic cost by representing weights or activations with fewer bits [4, 6]. Quantization-aware training (QAT) approaches such as DoReFa-Net and PACT enable low-bit training through differentiable approximations of the quantization function [7, 16]. Learned Step Size Quantization (LSQ) learns quantization step sizes during training
110 and improves low-bit quantization performance [17]. HAWQ and HAWQ-V2 use second-order sensitivity information to assign different bit-widths to different layers, showing that mixed-precision quantization can preserve accuracy more effectively than uniform quantization [18, 19]. These methods show that bit-width sensitivity is layer-dependent, motivating the learnable
115 quantization approach used in GETA. CAKR is complementary to these quantization methods because it focuses on teacher-guided recovery during GETA-based joint compression rather than changing the quantization scheme.

2.3. Joint Pruning and Quantization

Applying pruning and quantization separately can be suboptimal because
120 pruning changes the network structure and may alter quantization sensitivity. Zhang et al. showed that structured channel pruning and uniform quantization should be considered together rather than as independent post-processing steps [8]. GETA extends this direction by automatically combining structured pruning and quantization-aware training with explicit sparsity and bit-width
125 constraints through a quantization aware dependency graph and a quantization aware structured sparse optimizer [9]. In this paper, GETA is used as the common compression backbone for all evaluated methods; the GETA algorithm itself is not modified, and the compression schedule is held fixed within each dataset so that observed differences can be attributed to the
130 recovery objective.

2.4. Knowledge Distillation for Compressed Models

Knowledge distillation transfers soft output information from a teacher model to a student network [20]. Feature distillation methods such as FitNets, RKD, CRD, and PKT further transfer hidden-layer information
135 and representation geometry [21, 22, 23, 24]. These methods are relevant

because CAKR optionally uses a small feature distillation term. Several compression pipelines combine pruning, quantization, and KD to improve recovery: PQK [10] constructs a teacher from pruned weights in a two-phase pipeline, QRPK [11] studies quantization robustness with KD across multiple bit-widths, and a CAKD-based pipeline [12] combines pruning, QAT, and distillation. PDC proposes a one-stage joint framework for CNN acceleration that reconstructs redundant filters using intra-layer similarity and guides the process with teacher supervision [25]. Malihi and Heidemann investigate the combined effects of KD and pruning strategies on CIFAR-10 and CIFAR-100 across ResNet, DenseNet, EfficientNet, and MobileNet, finding that the interaction between pruning method and KD affects compression efficiency in an architecture-dependent manner [26]. Cui et al. combine structured pruning and dense knowledge distillation in an end-to-end joint training process for transformer compression, showing that joint optimization outperforms sequential application of the two techniques [27].

Unlike these methods, CAKR does not require a multi-stage pipeline, an internal teacher, or a fixed distillation approach. It uses the dense pretrained checkpoint as the teacher and adapts the KD signal according to the current compression state during GETA training. Unlike Cui et al. [27], who study transformer language models with a fixed dense KD signal, this work focuses on CNN-based joint structured pruning and weight quantization using GETA and provides a systematic multi-seed, statistically tested comparison of fixed and compression-aware KD strategies within this compression framework, across nine architecture-dataset combinations rather than a single setting. The architecture-dependent finding of Malihi and Heidemann [26] - that no single pruning-KD pairing is optimal across all architectures - is consistent with our results; our findings extend this by showing that the pattern also does not reduce to model size alone once dataset difficulty is varied jointly with architecture, and by providing a mechanistic account, grounded in CAKR’s own compression-risk gating behaviour, for why an adaptive strategy can underperform a fixed one in specific regimes.

3. Background

3.1. GETA Joint Structured Pruning and Quantization

GETA compresses a student network $f_s(x; \theta_s)$ by solving a constrained optimization problem over the weight parameters x and a set of learnable quantization parameters (d, q_m, t) , where q_m is the maximum mapped value, t

controls the shape of the quantization mapping, and d is the quantization step size [9]. Given a task loss $f(x, d, q_m, t)$ (cross-entropy alone for GETA-only, or the recovery losses introduced below for the KD-based methods), GETA
175 solves

$$\min_{\substack{x \in \mathbb{R}^n \\ (d, q_m, t)}} f(x, d, q_m, t) \quad (1)$$

$$\text{s.t.} \quad \text{Card}\{g \in \mathcal{G} \mid [x]_g = 0\} = K, \quad (2)$$

$$b_i \in [b_l, b_u], \quad i \in \mathcal{L}, \quad (3)$$

where $\mathcal{G}$ is the set of structured parameter groups identified by GETA’s quantization-aware dependency graph (QADG), K is the target sparsity level, and $[b_l, b_u]$ is the allowed layerwise bit-width range over the quantized layer index set $\mathcal{L}$. The bit width b_i for layer i is determined by the learned
180 quantization parameters as $b_i = \log_2((q_m)^t/d + 1) + 1$.

GETA solves Eqs. (1)-(3) using its quantization-aware structured sparse optimizer (QASSO), which updates x with search direction

$$s(x) = \begin{cases} -\alpha[\hat{\nabla}_x f]_g, & g \in \mathcal{G}_I, \\ -\alpha[\hat{\nabla}_x f]_g - \gamma[x^Q]_g, & g \in \mathcal{G}_R, \end{cases} \quad (4)$$

where $\hat{\nabla}_x f$ is the stochastic gradient of the task loss f with respect to x , $\mathcal{G}_I$ and $\mathcal{G}_R$ are the important and redundant parameter groups (selected
185 by a saliency criterion), x^Q is the quantized counterpart of x , and γ is a forget rate that progressively zeroes redundant groups while remaining aware of the quantization state. This is executed through four training stages - warm-up, projection (progressively tightening the bit-width range), joint (simultaneously pruning $\mathcal{G}_R$ and updating d, q_m, t), and cool-down - after
190 which GETA constructs the final compressed subnet by removing the zeroed groups and fixing the learned quantization parameters.

This paper does not modify GETA. The compression schedule (target sparsity, bit-width range, stage lengths) is held fixed across all three evaluated methods. What differs between GETA-only, fixed vanilla KD, and CAKR
195 is only the task loss $f(x, d, q_m, t)$ that is substituted into Eq. (1), which in turn is what $\hat{\nabla}_x f$ in Eq. (4) is computed from at every SGD step. In other words, KD and CAKR do not run alongside GETA as a separate process; they change the objective GETA’s own optimizer is minimizing, and consequently

the gradient signal that drives both the important-group updates and the
 200 forget-rate mechanism in Eq. (4).

3.2. GETA-Only Compression Objective

For GETA-only, the task loss substituted into Eq. (1) is cross-entropy
 alone:

$$f_{\text{GETA}}(x, d, q_m, t) = \mathcal{L}_{\text{CE}}(y, f_s(x)). \quad (5)$$

This is the primary baseline: GETA performs joint structured pruning and
 205 weight-only quantization exactly as described in Section 3.1, with no teacher
 signal contributing to f .

3.3. Vanilla Knowledge Distillation

Let z_t and z_s be teacher and student logits. Vanilla KD adds a distillation
 term to the task loss that GETA optimizes. The distillation loss itself
 210 minimizes the KL divergence between softened distributions [20], as defined
 in Eq. (6):

$$\mathcal{L}_{\text{KD}} = D_{\text{KL}}\left(\sigma\left(\frac{z_t}{T}\right) \parallel \sigma\left(\frac{z_s}{T}\right)\right), \quad (6)$$

where $\sigma(\cdot)$ denotes the softmax function and T is the temperature. The full
 task loss substituted into Eq. (1) for the fixed vanilla KD method is therefore

$$f_{\text{VKD}}(x, d, q_m, t) = \mathcal{L}_{\text{CE}} + \lambda_{\text{KD}} T^2 \mathcal{L}_{\text{KD}}. \quad (7)$$

The temperature T in Eq. (6) controls the softness of the output distributions
 215 - higher T produces softer distributions that reveal more inter-class similarity
 information from the teacher. The T^2 scaling factor in Eq. (7) preserves
 the gradient magnitude relative to the cross-entropy term. In vanilla KD,
 both T and λ_{KD} in Eq. (7) are fixed throughout training regardless of the
 student’s current state. Crucially, f_{VKD} is what GETA’s QASSO optimizer
 220 differentiates in Eq. (4): the KD term therefore influences not only the
 student’s weights directly, but also which groups GETA identifies as redundant
 and how aggressively they are forgotten, since $\hat{\nabla}_x f$ now carries gradient
 information from the teacher signal as well as the labels. Section 4 describes
 how vanilla KD and the proposed CAKR regularizer are applied on top of
 225 this GETA backbone.

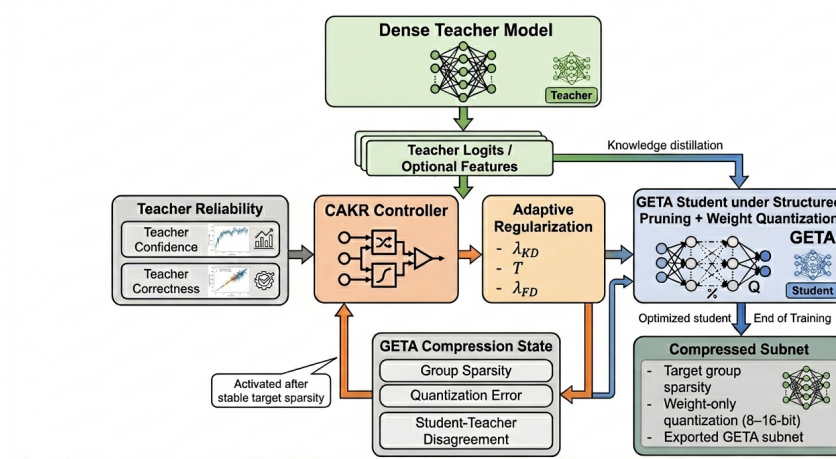


Figure 1: Overview of the GETA+CAKR framework. GETA performs joint structured pruning and weight-only quantization via QASSO (Eqs. (1)-(4)); CAKR substitutes its adaptive loss (Eq. (8)) as the task objective f that QASSO differentiates, and additionally reads GETA’s own compression-state indicators (group sparsity, quantization error) to control the teacher signal.

4. Teacher-Guided GETA Recovery

4.1. Overview

Two teacher-guided recovery strategies are compared on top of GETA: fixed vanilla KD and CAKR. In both cases, GETA’s compression procedure (Section 3.1) is unchanged; only the task loss f that GETA optimizes in Eq. (1) differs, as does the gradient $\hat{\nabla}_x f$ that consequently drives the QASSO search direction in Eq. (4). The dense checkpoint is used to initialize the student and, for KD-based methods, as the teacher. As illustrated in Fig. 1, CAKR activates after the target sparsity becomes stable and adapts the teacher signal using both teacher reliability and compression-state indicators drawn directly from GETA’s own internal state (group sparsity, quantization error).

4.2. Fixed Vanilla KD Recovery

The fixed KD baseline substitutes f_{VKD} (Eq. (7)) as the task loss GETA optimizes, applying a constant teacher signal after the target sparsity has stabilized for a predefined number of epochs, with $T = 3.0$ and a fixed λ_{KD} selected by dataset: 0.06 for CIFAR-10 and 0.08 for CIFAR-100 and TinyImageNet-200. This baseline tests whether simple, unconditional teacher

guidance is already sufficient for GETA recovery, and serves as an important
 245 comparison point for CAKR rather than a weak ablation.

4.3. Compression-Aware Knowledge Regularization (CAKR)

CAKR substitutes a different task loss into Eq. (1), keeping cross-entropy as the primary term and adding adaptive teacher-guided regularization using the teacher-correct weighted KD term defined in Section 4.4:

$$f_{\text{CAKR}}(x, d, q_m, t) = \mathcal{L}_{\text{CE}} + \lambda_{\text{KD}} T^2 \mathcal{L}_{\text{KD}}^w + \lambda_{\text{FD}} \mathcal{L}_{\text{FD}}. \quad (8)$$

250 Here $\mathcal{L}_{\text{CE}}$ is the supervised cross-entropy loss, $\mathcal{L}_{\text{KD}}^w$ is the teacher-correct confidence-weighted KD loss, $\mathcal{L}_{\text{FD}}$ is the optional feature distillation loss, T is the distillation temperature, and λ_{KD} and λ_{FD} control the KD and FD contributions. The CE term is not down-weighted. Because f_{CAKR} is the objective substituted into Eq. (1), GETA’s QASSO optimizer computes
 255 $\hat{\nabla}_x f_{\text{CAKR}}$ at every step in Eq. (4) - meaning the adaptive KD and FD terms directly shape which groups are pruned and how the forget rate γ evolves, not merely the final weight values after compression is otherwise complete. CAKR is activated only after the target sparsity is reached and stable, so that teacher guidance does not disturb the early GETA pruning and quantization
 260 search described in Section 3.1.

4.4. Teacher-Correct Confidence Weighting

To reduce negative transfer, CAKR emphasizes teacher guidance only when the teacher is correct and confident. For each training sample i , teacher confidence and correctness are defined as:

$$c_i = \max \sigma(z_t^i), \quad r_i = \mathbf{1}[\arg \max(z_t^i) = y_i], \quad w_i = r_i \cdot c_i. \quad (9)$$

265 The teacher-correct weighted KD loss is:

$$\mathcal{L}_{\text{KD}}^w = \frac{\sum_i w_i D_{\text{KL}}\left(\sigma\left(\frac{z_t^i}{T}\right) \parallel \sigma\left(\frac{z_s^i}{T}\right)\right)}{\sum_i w_i + \varepsilon}. \quad (10)$$

This design suppresses KD contributions from incorrect teacher predictions. As shown in Section 6, this masking becomes more consequential on harder datasets, where a larger fraction of teacher predictions are themselves incorrect, and interacts with the compression-risk gating described next to determine
 270 the realized strength of the teacher signal that reaches GETA’s optimizer.

4.5. Compression-State Controller

CAKR estimates a compression risk score using GETA’s own internal state during training: the current group sparsity G (the fraction of $\mathcal{G}_R$ already zeroed by GETA’s forget-rate mechanism, Eq. (4)), normalized quantization error Q (computed from GETA’s learned d, q_m, t), and teacher-student disagreement D :

$$R = w_1 G + w_2 Q + w_3 D, \quad w_1 = 0.35, \quad w_2 = 0.30, \quad w_3 = 0.10. \quad (11)$$

The KD coefficient and temperature are controlled as:

$$\lambda_{\text{KD}} = \lambda_{\text{KD}}^{\max} \cdot g_{\text{stage}} \cdot g_{\text{conf}} \cdot (1 - R), \quad (12)$$

$$T = T_{\min} + (T_{\max} - T_{\min}) \cdot R. \quad (13)$$

Here $g_{\text{stage}} = 0$ before GETA’s target sparsity is stable and $g_{\text{stage}} = 1$ afterward; $g_{\text{conf}} = \frac{1}{N} \sum_i c_i$ is the batch-averaged teacher confidence. Because R increases with GETA’s own group sparsity, quantization error, and teacher-student disagreement, and all three tend to remain elevated for longer on harder datasets, the realized λ_{KD} can be suppressed well below $\lambda_{\text{KD}}^{\max}$ for a substantial portion of training in exactly the settings where a stronger signal might otherwise help; this mechanism is examined further in Section 6.5. A small feature distillation term is optionally added:

$$\mathcal{L}_{\text{FD}} = \sum_l \beta_l \left\| \hat{h}_s^l - \hat{h}_t^l \right\|_2^2, \quad (14)$$

where $\hat{h}^l$ denotes normalized features at layer l , with maximum weight $\beta_l = 0.003$. The maximum KD weight $\lambda_{\text{KD}}^{\max}$ is selected per model-dataset pair from $\{0.06, 0.08\}$ and then fixed across all seeds of that setting.

4.6. Training Procedure

The training procedure is: (1) train a dense model and save the best checkpoint; (2) initialize the GETA student from the dense checkpoint; (3) use the same dense checkpoint as the teacher for KD-based methods; (4) run GETA compression with f_{GETA} (Eq. (5)) as the task loss during warm-up and early compression stages, so that QASSO’s search direction (Eq. (4)) is driven by cross-entropy alone; (5) compute compression risk R (Eq. (11)) from GETA’s own group sparsity and quantization state, teacher confidence g_{conf} , and teacher correctness, once target sparsity is stable; (6) substitute f_{VKD} (Eq. (7)) or f_{CAKR} (Eq. (8)) as the task loss and continue QASSO

optimization with this new objective; and (7) select the best checkpoint
 300 after stable target sparsity and export the compressed subnet via GETA’s
`construct_subnet()`.

5. Experimental Setup

Experiments are conducted on CIFAR-10 and CIFAR-100 [28] using
 ResNet20, ResNet56 [29], and VGG7 [30]. CIFAR-10 provides a standard
 305 10-class benchmark, while CIFAR-100 provides a harder 100-class setting
 that causes larger performance degradation after compression. ResNet20 and
 ResNet56 evaluate residual networks of different depths; VGG7 is included as a
 non-residual CNN. To test whether any pattern observed on CIFAR generalizes
 to a larger and more challenging dataset, we additionally evaluate all three
 310 architectures on TinyImageNet-200 [28], a 200-class dataset of 64×64 images
 (100K training, 10K validation images), using the same compression protocol
 as the CIFAR experiments, giving nine architecture-dataset combinations in
 total.

All compressed models use GETA with 60% target group sparsity and
 315 weight-only quantization in the 8-16-bit range. CIFAR-10 and CIFAR-100
 experiments use 150 compression epochs, while TinyImageNet-200 experiments
 use 100 compression epochs across all three architectures, reflecting the higher
 per-epoch computational cost of the larger dataset; this schedule is applied
 identically to GETA-only, vanilla KD, and CAKR within each dataset. All
 320 settings use batch size 128, SGD with learning rate 0.05, momentum 0.9,
 weight decay 5×10^{-4} , gradient clipping 1.0, 10 warm-up epochs, and 50 ramp
 epochs. The best checkpoint is selected only after the target sparsity is reached
 and stable for 10 epochs. Main results are averaged over seeds 123, 234, and
 345, reported as mean $\pm$ standard deviation. For each KD-based method, a
 325 paired t -test against the GETA-only baseline is additionally computed per
 setting, together with Cohen’s d effect size. Given the small sample size,
 results are interpreted using both the significance test and the consistency of
 direction across seeds, rather than treating $p < 0.05$ as the sole criterion for a
 method having “worked.”

330 For vanilla KD, $T = 3.0$ is fixed for all settings, while λ_{KD} is set to
 0.06 for CIFAR-10 and 0.08 for CIFAR-100 and TinyImageNet-200. For
 CAKR, $\lambda_{\text{KD}}^{\text{max}}$ is selected from $\{0.06, 0.08\}$ per model-dataset pair, with the
 feature distillation term either disabled or used with maximum weight 0.003.

Table 1: Dense baseline metrics used for student initialization and as teacher models, across all nine architecture-dataset combinations.

Model / Dataset	Dense Acc. (%)	Params	MACs
ResNet20 / CIFAR-10	92.96	272.47K	41.00M
ResNet56 / CIFAR-10	94.29	855.77K	125.75M
VGG7 / CIFAR-10	92.62	1.15M	606.00M
ResNet20 / CIFAR-100	69.14	278.32K	40.82M
ResNet56 / CIFAR-100	73.26	861.62K	125.75M
VGG7 / CIFAR-100	71.65	1.17M	152.79M
ResNet20 / TinyImageNet-200	52.02	284.82K	163.26M
ResNet56 / TinyImageNet-200	58.73	868.12K	503.00M
VGG7 / TinyImageNet-200	55.79	4.68M	2430.18M

Controller weights are fixed at $w_1 = 0.35$, $w_2 = 0.30$, $w_3 = 0.10$ across all settings.

All experiments were conducted on Kaggle using $2 \times$ NVIDIA Tesla T4 GPUs with PyTorch 1.13.1 and torchvision 0.14.1. Top-1 accuracy is the primary metric. Test loss, MACs, and nonzero parameters are also reported. Table 1 shows the dense baseline metrics for all nine settings. All training logs and per-seed metrics are available at <https://github.com/sabbir211/CAKR>.

6. Results and Discussion

This section compares GETA-only, GETA+vanilla KD, and GETA+CAKR under the same compression protocol across all nine settings. The evaluation covers ResNet20, ResNet56, and VGG7 on CIFAR-10, CIFAR-100, and TinyImageNet-200, allowing the effects of model capacity and dataset difficulty to be examined systematically. All results are reported as mean $\pm$ standard deviation over three seeds to capture both average performance and cross-seed variability. In addition to accuracy, test loss, MACs, and parameter counts are considered to assess the trade-off between predictive performance and compression efficiency. Every KD-based method’s improvement over GETA-only is further evaluated using a paired t -test, with the corresponding p -value and Cohen’s d effect size reported to determine both statistical significance and the magnitude of the observed improvement.

Table 2 shows the compression damage and teacher-guided recovery for all

Table 2: Compression damage and teacher-guided recovery across all nine settings (mean accuracy, %). KD Recovery and CAKR Recovery are gains over the GETA-only baseline. [†]Method falls below the GETA-only baseline. Bold marks the larger gain in each row.

Model / Dataset	Dense	GETA-only	KD Recovery	CAKR Recovery
ResNet20 / CIFAR-10	92.96	87.90 (−5.06)	87.64 (−0.26 [†])	88.05 (+0.15)
ResNet56 / CIFAR-10	94.29	93.15 (−1.14)	93.22 (+0.07)	93.20 (+0.05)
VGG7 / CIFAR-10	92.62	90.75 (−1.87)	91.09 (+0.34)	90.88 (+0.13)
ResNet20 / CIFAR-100	69.14	50.91 (−18.23)	51.44 (+0.53)	51.74 (+0.83)
ResNet56 / CIFAR-100	73.26	67.68 (−5.58)	68.59 (+0.91)	68.21 (+0.53)
VGG7 / CIFAR-100	71.65	63.57 (−8.08)	64.08 (+0.51)	63.98 (+0.41)
ResNet20 / TinyImageNet-200	52.02	32.44 (−19.58)	33.49 (+1.05)	32.28 (−0.17 [†])
ResNet56 / TinyImageNet-200	58.73	50.40 (−8.33)	51.23 (+0.83)	50.72 (+0.32)
VGG7 / TinyImageNet-200	55.79	53.75 (−2.04)	54.58 (+0.83)	54.15 (+0.40)

355 nine evaluated settings. CIFAR-100 and TinyImageNet-200 settings generally suffer larger compression damage than CIFAR-10. The largest compression damage observed in this study occurs on ResNet20/TinyImageNet-200, where GETA-only accuracy drops by 19.58 percentage points from its 52.02% dense baseline; among the CIFAR settings, ResNet20/CIFAR-100 shows the largest
360 damage, losing 18.23 percentage points from its dense baseline under GETA compression alone.

Table 3 reports the paired t -test and Cohen’s d effect size for each KD-based method against GETA-only, for all eighteen method-setting comparisons (all nine settings, both methods), each with fully verified per-seed source data.
365 Even at $n = 3$, the settings that reach significance do so with moderate-to-large effect sizes (Cohen’s d from 3.2 to 5.5), indicating these are not marginal, noise-driven results; conversely, several non-significant comparisons correspond to genuinely small mean differences, consistent with those settings being close to the dense baseline’s accuracy ceiling. Notably, the two TinyImageNet-200
370 settings with the largest raw point-gains for vanilla KD (ResNet20 and ResNet56) do not reach significance once verified per-seed GETA-only data is used, underscoring why direction of effect and significance should be read together rather than significance alone.

Table 4 reports the main accuracy comparison across all nine settings. Both
375 KD-based methods often improve over GETA-only, but neither dominates uniformly, and the pattern does not reduce to a simple rule based on architecture size. Of the eighteen method-setting comparisons, four reach

Table 3: Paired t -test results ($n = 3$ seeds) and Cohen’s d effect size for each KD-based method against the GETA-only baseline. Bold rows reach $p < 0.05$.

Model / Dataset	Comparison	Mean Δ (pp)	t	p	Cohen’s d
ResNet20 / CIFAR-10	Vanilla KD vs. GETA-only	−0.26	−0.92	0.454	−0.53
	CAKR vs. GETA-only	+0.15	0.54	0.643	0.31
ResNet20 / CIFAR-100	Vanilla KD vs. GETA-only	+0.53	2.97	0.097	1.71
	CAKR vs. GETA-only	+0.83	5.52	0.031	3.19
ResNet20 / TinyImageNet-200	Vanilla KD vs. GETA-only	+1.05	2.08	0.173	1.20
	CAKR vs. GETA-only	−0.17	−0.81	0.505	−0.47
ResNet56 / CIFAR-10	Vanilla KD vs. GETA-only	+0.07	0.86	0.483	0.49
	CAKR vs. GETA-only	+0.05	0.61	0.602	0.35
VGG7 / CIFAR-10	Vanilla KD vs. GETA-only	+0.34	6.11	0.026	3.53
	CAKR vs. GETA-only	+0.14	1.41	0.294	0.81
ResNet56 / CIFAR-100	Vanilla KD vs. GETA-only	+0.92	9.53	0.011	5.50
	CAKR vs. GETA-only	+0.53	2.86	0.103	1.65
VGG7 / CIFAR-100	Vanilla KD vs. GETA-only	+0.51	2.71	0.113	1.57
	CAKR vs. GETA-only	+0.41	1.81	0.212	1.04
ResNet56 / TinyImageNet-200	Vanilla KD vs. GETA-only	+0.83	3.24	0.083	1.87
	CAKR vs. GETA-only	+0.32	1.20	0.352	0.70
VGG7 / TinyImageNet-200	Vanilla KD vs. GETA-only	+0.83	6.91	0.020	3.99
	CAKR vs. GETA-only	+0.41	2.22	0.157	1.28

significance: one for CAKR and three for vanilla KD. Vanilla KD also shows the largest point-gain of any setting on ResNet20/TinyImageNet-200 without reaching significance, while CAKR does not exceed GETA-only in that same setting - the one case in this study where CAKR’s mean accuracy falls below the baseline outright.

6.1. CIFAR-100 Results

Table 5 provides the detailed CIFAR-100 results. Fig. 2 visualizes the top-1 accuracy comparison. CIFAR-100 is an important benchmark because its 100 fine-grained classes cause larger accuracy degradation after compression than CIFAR-10, giving more room for a teacher signal to matter.

On ResNet20/CIFAR-100, CAKR is best in accuracy ($51.74 \pm 0.60\%$) and test loss (1.7585 ± 0.0152), and this is the only setting in the study where CAKR’s improvement over GETA-only reaches significance ($p = 0.031$). Vanilla KD also improves over GETA-only (51.44%, +0.53 pp) but does

Table 4: Main accuracy comparison across all nine settings (mean $\pm$ std, 3 seeds). [†]Falls below the GETA-only baseline. “Sig.” indicates which method, if any, reaches $p < 0.05$ against GETA-only.

Model / Dataset	GETA-only (%)	Vanilla KD (%)	CAKR (%)	Sig.
ResNet20 / CIFAR-10	87.90 $\pm$ 0.61	87.64 $\pm$ 0.34 [†]	88.05 $\pm$ 0.13	neither
ResNet56 / CIFAR-10	93.15 $\pm$ 0.19	93.22 $\pm$ 0.05	93.20 $\pm$ 0.05	neither
VGG7 / CIFAR-10	90.75 $\pm$ 0.22	91.09 $\pm$ 0.13	90.88 $\pm$ 0.20	Vanilla KD
ResNet20 / CIFAR-100	50.91 $\pm$ 0.83	51.44 $\pm$ 0.67	51.74 $\pm$ 0.60	CAKR
ResNet56 / CIFAR-100	67.68 $\pm$ 0.26	68.59 $\pm$ 0.09	68.21 $\pm$ 0.17	Vanilla KD
VGG7 / CIFAR-100	63.57 $\pm$ 0.24	64.08 $\pm$ 0.46	63.98 $\pm$ 0.21	neither
ResNet20 / TinyImageNet-200	32.44 $\pm$ 0.64	33.49 $\pm$ 0.49	32.28 $\pm$ 0.32 [†]	neither
ResNet56 / TinyImageNet-200	50.40 $\pm$ 0.27	51.23 $\pm$ 0.30	50.72 $\pm$ 0.29	neither
VGG7 / TinyImageNet-200	53.75 $\pm$ 0.37	54.58 $\pm$ 0.31	54.15 $\pm$ 0.09	Vanilla KD

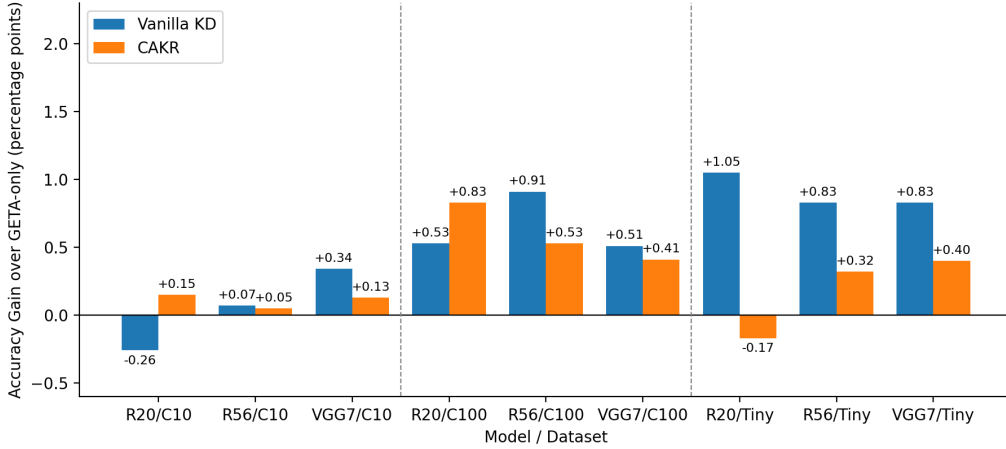


Figure 2: Accuracy comparison across all nine CIFAR and TinyImageNet settings (mean $\pm$ std, 3 seeds). The annotation marks where vanilla KD falls below GETA-only.

not reach significance ($p = 0.097$). On ResNet56/CIFAR-100, the pattern reverses: vanilla KD achieves the highest accuracy ($68.59 \pm 0.09\%$), lower loss, and lower MACs, with a statistically significant gain over GETA-only ($p = 0.011$), while CAKR’s gain ($+0.53\%$) does not reach significance ($p = 0.103$). On VGG7/CIFAR-100, vanilla KD achieves the highest mean accuracy ($64.08 \pm 0.46\%$ vs. $63.98 \pm 0.21\%$ for CAKR), but neither method’s gain over GETA-only reaches significance in this setting (vanilla KD $p = 0.113$; CAKR $p = 0.212$).

Table 5: Detailed CIFAR-100 compression results (mean $\pm$ std, 3 seeds).

Model	Method	Acc. (%)	Test Loss	MACs	Params
ResNet20	GETA-only	50.91 $\pm$ 0.83	1.7849 $\pm$ 0.0371	9.95 $\pm$ 0.45M	44.33 $\pm$ 1.03K
	Vanilla KD	51.44 $\pm$ 0.67	1.7673 $\pm$ 0.0328	9.80 $\pm$ 0.26M	43.79 $\pm$ 1.96K
	CAKR	51.74 $\pm$ 0.60	1.7585 $\pm$ 0.0152	10.23 $\pm$ 0.29M	44.37 $\pm$ 1.91K
ResNet56	GETA-only	67.68 $\pm$ 0.26	1.1641 $\pm$ 0.0193	40.67 $\pm$ 0.76M	267.36 $\pm$ 1.60K
	Vanilla KD	68.59 $\pm$ 0.09	1.1589 $\pm$ 0.0034	39.70 $\pm$ 0.47M	267.75 $\pm$ 2.72K
	CAKR	68.21 $\pm$ 0.17	1.1616 $\pm$ 0.0210	39.80 $\pm$ 1.23M	265.77 $\pm$ 2.51K
VGG7	GETA-only	63.57 $\pm$ 0.24	1.349 $\pm$ 0.004	27.83 $\pm$ 0.49M	157.35 $\pm$ 1.45K
	Vanilla KD	64.08 $\pm$ 0.46	1.290 $\pm$ 0.015	28.84 $\pm$ 0.75M	160.63 $\pm$ 3.31K
	CAKR	63.98 $\pm$ 0.21	1.291 $\pm$ 0.004	29.10 $\pm$ 0.80M	160.06 $\pm$ 2.87K

6.2. CIFAR-10 Results

Table 6 shows the CIFAR-10 results. Gains are smaller and less consistent on CIFAR-10 than on the harder datasets, because GETA-only already achieves high compressed accuracy, leaving less headroom for a teacher signal to improve upon. Vanilla KD falls slightly below GETA-only on ResNet20 (a non-significant difference), while CAKR improves both accuracy and stability in that same setting, reducing the standard deviation from ± 0.61 to ± 0.13 . For ResNet56/CIFAR-10, vanilla KD and CAKR achieve very similar accuracy to each other and to GETA-only (all within 0.07 pp), and neither difference is significant. For VGG7/CIFAR-10, vanilla KD achieves the highest mean accuracy (91.09%) with a statistically significant improvement over GETA-only ($p = 0.026$), while CAKR (90.88%) also improves over GETA-only but without reaching significance ($p = 0.294$).

6.3. TinyImageNet-200 Results

To test whether patterns observed on CIFAR generalize to a larger and more challenging dataset, all three architectures are evaluated on TinyImageNet (200 classes, 64 $\times$ 64 images) using the same compression protocol and three seeds. Table 7 reports the results.

Vanilla KD achieves the highest mean accuracy in all three TinyImageNet-200 settings. The gain is largest for ResNet20 (+1.05 pp), followed by ResNet56 and VGG7 (both +0.83 pp), but only the VGG7 gain reaches statistical significance ($p = 0.020$); the ResNet20 gain, despite being the largest in absolute terms, does not ($p = 0.173$), nor does the ResNet56 gain ($p = 0.083$),

Table 6: Detailed CIFAR-10 compression results (mean $\pm$ std, 3 seeds).

Model	Method	Acc. (%)	Test Loss	MACs	Params
ResNet20	GETA-only	87.90 $\pm$ 0.61	0.370 $\pm$ 0.016	14.86 $\pm$ 0.62M	40.23K
	Vanilla KD	87.64 $\pm$ 0.34	0.407 $\pm$ 0.007	14.05 $\pm$ 0.38M	40.01 $\pm$ 0.66K
	CAKR	88.05 $\pm$ 0.13	0.390 $\pm$ 0.007	14.44 $\pm$ 0.58M	41.38K
ResNet56	GETA-only	93.15 $\pm$ 0.19	0.268 $\pm$ 0.007	56.79 $\pm$ 0.51M	212.94K
	Vanilla KD	93.22 $\pm$ 0.05	0.300 $\pm$ 0.002	56.86 $\pm$ 0.44M	212.46 $\pm$ 1.36K
	CAKR	93.20 $\pm$ 0.05	0.289 $\pm$ 0.013	56.63 $\pm$ 0.36M	213.90K
VGG7	GETA-only	90.75 $\pm$ 0.22	0.301 $\pm$ 0.001	41.22 $\pm$ 1.27M	161.01 $\pm$ 3.33K
	Vanilla KD	91.09 $\pm$ 0.13	0.341 $\pm$ 0.007	42.23 $\pm$ 1.44M	159.70 $\pm$ 1.73K
	CAKR	90.88 $\pm$ 0.20	0.318 $\pm$ 0.006	41.47 $\pm$ 1.70M	160.92 $\pm$ 0.96K

Table 7: TinyImageNet-200 compression results across all three architectures (mean $\pm$ std, 3 seeds). [†]Falls below the GETA-only baseline.

Model	Method	Acc. (%)	Test Loss	MACs	Params
ResNet20	GETA-only	32.44 $\pm$ 0.64	2.8687 $\pm$ 0.0310	36.71 $\pm$ 3.05M	51.56 $\pm$ 1.88K
	Vanilla KD	33.49 $\pm$ 0.49	2.8278 $\pm$ 0.0316	38.33 $\pm$ 1.94M	51.42 $\pm$ 0.68K
	CAKR	32.28 $\pm$ 0.32 [†]	2.8660 $\pm$ 0.0147	38.36 $\pm$ 2.81M	49.84 $\pm$ 1.06K
ResNet56	GETA-only	50.40 $\pm$ 0.27	2.0381 $\pm$ 0.0041	503.00 $\pm$ 0.00M	563.13 $\pm$ 4.96K
	Vanilla KD	51.23 $\pm$ 0.30	1.9879 $\pm$ 0.0069	503.00 $\pm$ 0.00M	563.54 $\pm$ 4.98K
	CAKR	50.72 $\pm$ 0.29	2.0153 $\pm$ 0.0101	503.00 $\pm$ 0.00M	558.75 $\pm$ 4.21K
VGG7	GETA-only	53.75 $\pm$ 0.37	2.0033 $\pm$ 0.0148	328.59 $\pm$ 4.38M	780.90 $\pm$ 13.34K
	Vanilla KD	54.58 $\pm$ 0.31	1.9207 $\pm$ 0.0131	330.82 $\pm$ 2.45M	781.51 $\pm$ 9.08K
	CAKR	54.15 $\pm$ 0.09	1.9551 $\pm$ 0.0058	331.09 $\pm$ 3.99M	778.07 $\pm$ 9.23K

reflecting the higher cross-seed variability of the ResNet20 result and the more modest point-gain of the ResNet56 result. CAKR improves over GETA-only on ResNet56 and VGG7, but by a smaller margin in both cases, and does not improve over GETA-only on ResNet20/TinyImageNet-200, where its mean accuracy is slightly below the baseline (32.28% vs. 32.44%). This is the one setting in the study where CAKR’s mean accuracy falls below GETA-only outright, rather than simply failing to reach significance while still showing a positive gain, and it occurs on the combination of the smallest architecture and the hardest dataset evaluated.

Table 8: Additive vs. interpolated vanilla KD on TinyImageNet-200 (mean $\pm$ std, 3 seeds). p -values are against the GETA-only baseline; interpolated-run values are reproduced from the original experiments and not recomputed here. Bold marks the larger gain in each row.

Model	GETA-only (%)	Additive KD (%)	p (additive)	Interpolated KD (%)	p (interp.)
ResNet20	32.44 $\pm$ 0.64	33.49 $\pm$ 0.49	0.173	33.61 $\pm$ 0.03	0.082
ResNet56	50.40 $\pm$ 0.27	51.23 $\pm$ 0.30	0.083	52.03 $\pm$ 0.38	0.006
VGG7	53.75 $\pm$ 0.37	54.58 $\pm$ 0.31	0.020	55.82 $\pm$ 0.27	0.016

6.4. Formulation Sensitivity: Additive vs. Interpolated KD on TinyImageNet-200

The vanilla KD results reported above for TinyImageNet-200 use the same additive formulation applied throughout this paper for CIFAR-10 and
435 CIFAR-100 (Eq. (7)), so that the KD loss form is held fixed across all nine settings. An earlier iteration of the TinyImageNet-200 experiments instead used an interpolated, Hinton-style [20] blend between the cross-entropy and distillation terms:

$$f_{\text{VKD}}^{\text{interp}}(x, d, q_m, t) = (1 - \alpha) \mathcal{L}_{\text{CE}} + \alpha T^2 \mathcal{L}_{\text{KD}}, \quad (15)$$

with $\alpha = 0.5$ and $T = 4.0$, giving an effective KD coefficient $\alpha T^2 = 8.0$
440 and a cross-entropy weight of only $1 - \alpha = 0.5$. This contrasts sharply with the additive form used for the main TinyImageNet-200 results, where $T^2 \lambda_{\text{KD}} = 0.72$ (Section 6.5) and cross-entropy retains its full weight of 1. Unlike the additive form, which adds a scaled KD term on top of a full-weight cross-entropy loss, the interpolated form down-weights cross-entropy as α
445 increases, so the two formulations place a materially different balance between supervised and teacher signal: the interpolated run applies both a substantially stronger teacher signal and a substantially weaker supervised signal than the additive run.

Table 8 reports both formulations side by side for all three TinyImageNet-200
450 architectures, using the interpolated results and significance values exactly as computed against the GETA-only baseline in the original run. Interpolated KD yields larger point-gains than additive KD in all three settings, and reaches significance in two of three (ResNet56, $p = 0.006$; VGG7, $p = 0.016$), compared with one of three for additive KD (VGG7 only, $p = 0.020$). ResNet20
455 does not reach significance under either formulation ($p = 0.082$ interpolated vs. $p = 0.173$ additive), though the interpolated gain is again larger.

This comparison suggests that, on the larger and harder TinyImageNet-200 setting, down-weighting cross-entropy in favor of the teacher signal (as the

interpolated form does) recovers more accuracy than adding an unweighted
 460 KD term on top of full-strength cross-entropy. It also indicates that the
 choice of KD loss formulation, and not only architecture, dataset difficulty, or
 compression severity, is itself a factor in how much teacher-guided recovery a
 GETA-compressed student can achieve. Because a systematic head-to-head
 comparison of the two formulations across all nine settings (rather than
 465 TinyImageNet-200 alone) was outside the scope of this study, the main
 results and conclusions elsewhere in this paper use the additive formulation
 throughout for consistency with CIFAR-10 and CIFAR-100, and the interpolated
 results are reported here as a supplementary finding motivating future work.

6.5. *Effect of Student Capacity, Dataset Difficulty, and Compression Severity*

470 Across the nine settings evaluated, vanilla KD’s significant improvements
 occur on ResNet56 (CIFAR-100 only) and on VGG7 (CIFAR-10, TinyImageNet
 200) - spanning both a larger architecture and a mid-sized one, and both
 an easier and a harder dataset. Notably, on ResNet56/TinyImageNet-200
 vanilla KD still improves over GETA-only in direction (+0.83 pp) but the
 475 gain does not reach significance once verified per-seed GETA-only data is
 used ($p = 0.083$), so ResNet56’s significant recovery in this study is confined
 to CIFAR-100. This pattern is not consistent with a rule stating that fixed
 KD only benefits large models; it is more consistent with fixed KD helping
 whenever the student retains sufficient post-compression capacity to absorb
 480 an unconditional, comparatively strong teacher signal ($T^2\lambda_{\text{KD}} = 0.54$ on
 CIFAR-10, 0.72 on CIFAR-100 and TinyImageNet-200) without that signal
 crowding out the cross-entropy objective, and whenever cross-seed variability
 is low enough for the gain to be distinguishable from noise at $n = 3$.

CAKR’s one significant improvement occurs on ResNet20/CIFAR-100
 485 - the smallest architecture in the study, paired with a moderately difficult
 dataset. However, on ResNet20/TinyImageNet-200 - the same, smallest
 architecture, paired with the hardest dataset - CAKR does not improve
 over GETA-only at all, while vanilla KD shows the largest directional gain
 observed for that setting. If student capacity alone determined which method
 was preferable, CAKR would be expected to perform at least as well as
 490 vanilla KD on ResNet20 regardless of dataset; this is not what is observed,
 and the discrepancy motivates a mechanistic explanation rather than an
 architecture-size rule.

CAKR’s effective KD coefficient, $\lambda_{\text{KD}} = \lambda_{\text{KD}}^{\text{max}} \cdot g_{\text{stage}} \cdot g_{\text{conf}} \cdot (1 - R)$ (Eq. (12)),
 495 is suppressed by the compression risk term R , which is driven substantially

by group sparsity once the target sparsity is reached and remains elevated for an extended portion of post-pruning training on harder datasets. In addition, the teacher-correct confidence weighting term $w_i = r_i \cdot c_i$ discards KD signal entirely for any sample the teacher misclassifies, and this fraction grows with task difficulty. On the combination of the hardest dataset and the smallest architecture, these two effects compound: the realized KD weight remains a small fraction of its configured maximum for most of training, and the compressed student receives only a weak, further-diluted teacher signal at precisely the point where a stronger signal appears to help, as evidenced by vanilla KD’s larger directional gain in the same setting. This mechanism, rather than architecture size in isolation, offers a more complete account of why CAKR’s one significant success (ResNet20/CIFAR-100) and its one outright shortfall (ResNet20/TinyImageNet-200) both occur on the same architecture: CAKR’s own gating strength, not the student’s capacity alone, determines whether the adaptive signal is strong enough to help.

CAKR does, however, provide a broader and more consistent benefit independent of mean-accuracy significance: a reduction in cross-seed standard deviation relative to GETA-only, most notably on ResNet20/CIFAR-10 ($\pm 0.61 \rightarrow \pm 0.13$). This indicates that CAKR’s self-limiting signal can improve the reliability of recovery across seeds even in settings where its mean-accuracy gain is not statistically distinguishable from GETA-only.

6.6. Limitations of the Evidence

Several limitations should be noted. First, the vanilla KD baseline uses only two fixed configurations: $T = 3.0$ with $\lambda_{\text{KD}} = 0.06$ for CIFAR-10 and $\lambda_{\text{KD}} = 0.08$ for CIFAR-100 and TinyImageNet-200. More detailed per-model tuning of vanilla KD might change its performance in some settings; however, CAKR’s compression-state controller adapts the effective KD strength without manual per-setting tuning, which may still be a practical advantage independent of the specific mean-accuracy outcome. Second, results are limited to three seeds per setting; while this is sufficient to detect moderate-to-large effect sizes (as shown by the four significant comparisons identified across the study), it limits statistical power for smaller true effects, particularly on CIFAR-10 settings where both methods’ mean differences from GETA-only are small. A larger seed count, applied consistently rather than selectively to borderline settings, would strengthen these comparisons. Third, experiments remain restricted to CNNs; generalization to transformer architectures supported by GETA remains to be validated.

7. Conclusion

This paper studied teacher-guided recovery for GETA-based joint structured pruning and weight quantization. Fixed vanilla KD and a compression-aware adaptive variant (CAKR) were compared against GETA-only under the same compression protocol across ResNet20, ResNet56, and VGG7 on CIFAR-10, CIFAR-100, and TinyImageNet-200 - nine architecture-dataset combinations in total - with paired significance testing for every comparison.

Teacher guidance can improve GETA-compressed models, but this benefit is neither universal nor uniform across methods. Of eighteen method-setting comparisons, four reach statistical significance: one for CAKR (ResNet20 with CIFAR-100, $p = 0.031$) and three for vanilla KD (ResNet56/CIFAR-100, VGG7/CIFAR-10, and VGG7 on TinyImageNet-200). Vanilla KD falls slightly below the GETA-only baseline on ResNet20/CIFAR-10 (a non-significant difference), while CAKR achieves the highest mean accuracy and the largest stability improvement in that same setting ($\pm 0.61 \rightarrow \pm 0.13$). Conversely, on ResNet20/TinyImageNet-200, CAKR does not improve over GETA-only at all, while vanilla KD shows the largest directional gain observed for that setting. This pattern is not consistent with a simple rule in which adaptive KD favors small models and fixed KD favors large ones; instead, it is best explained by CAKR’s own severity-based gating, which suppresses the teacher signal as compression risk and task difficulty jointly increase, occasionally to the point of providing no net benefit.

Fixed KD and CAKR are therefore best understood as offering different trade-offs rather than one being categorically superior: fixed KD supplies a stronger, unconditional signal that produces the most frequent significant gains in this study, while CAKR supplies a self-limiting, compression-aware signal that produces one significant gain and a broader improvement in cross-seed stability, at the cost of occasionally under-delivering when compression severity and dataset difficulty are simultaneously high. Selecting between them, or designing a method that combines their strengths, requires accounting for the joint effect of architecture, dataset difficulty, and compression severity rather than any one factor in isolation.

Future work will extend this study to larger datasets such as ImageNet, transformer architectures supported by GETA, additional seeds on borderline significant settings (selected by effect size and compute cost rather than by proximity to $p = 0.05$, to avoid outcome-dependent sampling bias), automated selection of KD strength based on measured compression severity and dataset

570 difficulty rather than architecture size alone, and a systematic head-to-head
comparison of additive and interpolated KD formulations across all nine
architecture-dataset settings, motivated by the formulation-sensitivity result
observed on TinyImageNet-200 (Section 6.4).

Funding

575 This research did not receive any specific grant from funding agencies in
the public, commercial, or not-for-profit sectors.

Declaration of Competing Interest

The authors declare no competing financial interests.

Data Availability

580 All training logs, per-seed metrics, and figures are publicly available at
<https://github.com/sabbir211/CAKR>.

References

- [1] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand,
M. Andreetto, H. Adam, MobileNets: Efficient convolutional neural
585 networks for mobile vision applications, arXiv:1704.04861 (2017).
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen,
MobileNetV2: Inverted residuals and linear bottlenecks, in: Proc. CVPR,
2018, pp. 4510–4520.
- [3] S. Han, J. Pool, J. Tran, W. J. Dally, Learning both weights and
590 connections for efficient neural network, in: Proc. NeurIPS, 2015, pp.
1135–1143.
- [4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam,
D. Kalenichenko, Quantization and training of neural networks for
efficient integer-arithmetic-only inference, in: Proc. CVPR, 2018, pp.
595 2704–2713.

- [5] Y. He, L. Xiao, Structured pruning for deep convolutional neural networks: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (5) (2024) 2900–2919.
- [6] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio, Quantized neural networks: Training neural networks with low precision weights and activations, *J. Mach. Learn. Res.* 18 (187) (2018) 1–30.
- [7] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, Y. Zou, DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients, *arXiv:1606.06160* (2016).
- [8] X. Zhang, I. Colbert, S. Das, Learning low-precision structured subnetworks using joint layerwise channel pruning and uniform quantization, *Appl. Sci.* 12 (15) (2022) 7829. doi:10.3390/app12157829.
- [9] X. Qu, D. Aponte, C. Banbury, D. P. Robinson, T. Ding, K. Koishida, I. Zharkov, T. Chen, Automatic joint structured pruning and quantization for efficient neural network training and compression, in: *Proc. CVPR*, 2025, pp. 15234–15244.
- [10] J. Kim, S. Chang, N. Kwak, POK: Model compression via pruning, quantization, and knowledge distillation, in: *Proc. Interspeech*, 2021, pp. 4568–4572. doi:10.21437/Interspeech.2021-248.
- [11] J. Kim, Quantization robust pruning with knowledge distillation, *IEEE Access* 11 (2023) 26419–26426. doi:10.1109/ACCESS.2023.3257864.
- [12] W. Chae, K. Seo, Pipeline of pruning, knowledge distillation, and quantization for model compression, *J. Electr. Eng. Technol.* (2025). doi:10.1007/s42835-025-02541-7.
- [13] G. Fang, X. Ma, M. Song, M. B. Mi, X. Wang, DepGraph: Towards any structural pruning, in: *Proc. CVPR*, 2023, pp. 16091–16101.
- [14] T. Chen, B. Ji, T. Ding, B. Fang, G. Wang, Z. Zhu, L. Liang, Y. Shi, S. Yi, X. Tu, Only train once: A one-shot neural network training and pruning framework, in: *Proc. NeurIPS*, 2021.

- [15] T. Chen, L. Liang, T. Ding, I. Zharkov, Z. Zhu, OTOv2: Automatic, generic, user-friendly neural network training and compression, in: Proc. ICLR, 2023.
- [16] J. Choi, Z. Wang, S. Venkataramani, P. Chuang, V. Srinivasan, K. Gopalakrishnan, PACT: Parameterized clipping activation for quantized neural networks, arXiv:1805.06085 (2018).
- [17] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, D. S. Modha, Learned step size quantization, in: Proc. ICLR, 2020.
- [18] Z. Dong, Z. Yao, A. Gholami, M. Mahoney, K. Keutzer, HAWQ: Hessian aware quantization of neural networks with mixed-precision, in: Proc. ICCV, 2019, pp. 293–302.
- [19] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, K. Keutzer, HAWQ-V2: Hessian aware trace-weighted quantization of neural networks, in: Proc. NeurIPS, 2020.
- [20] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv:1503.02531 (2015).
- [21] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, Y. Bengio, FitNets: Hints for thin deep nets, in: Proc. ICLR, 2015.
- [22] W. Park, D. Kim, Y. Lu, M. Cho, Relational knowledge distillation, in: Proc. CVPR, 2019, pp. 3967–3976.
- [23] Y. Tian, D. Krishnan, P. Isola, Contrastive representation distillation, in: Proc. ICLR, 2020.
- [24] N. Passalis, A. Tefas, Learning deep representations with probabilistic knowledge transfer, in: Proc. ECCV Workshops, 2018.
- [25] T. Niu, Y. Teng, L. Jin, P. Zou, Y. Liu, Pruning-and-distillation: One-stage joint compression framework for CNNs via clustering, Image Vis. Comput. 136 (2023) 104738. doi:10.1016/j.imavis.2023.104738.
- [26] L. Malihi, G. Heidemann, Matching the ideal pruning method with knowledge distillation for optimal compression, Appl. Syst. Innov. 7 (4) (2024) 56. doi:10.3390/asi7040056.

- [27] B. Cui, Y. Li, Z. Zhang, Joint structured pruning and dense knowledge distillation for efficient transformer model compression, *Neurocomputing* 458 (2021) 56–69. doi:10.1016/j.neucom.2021.05.084.
- [28] A. Krizhevsky, Learning multiple layers of features from tiny images, 660 Tech. rep., University of Toronto (2009).
- [29] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proc. CVPR*, 2016, pp. 770–778.
- [30] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: *Proc. ICLR*, 2015.